

waveeProjectBW - dokumentacja techniczna

FULL

Wersja scalona podstron dokumentacji technicznej.

[01_architektura_i_przeplyw.html](#)

01. Architektura i przepływ danych

Opis sposobu komunikacji frontend-backend-baza danych-Spotify API.

Model działania systemu

1. Frontend inicjuje zapytanie HTTP do backendu.
2. Backend wykonuje middleware i logikę endpointu.
3. W razie potrzeby backend komunikuje się z PostgreSQL lub Spotify API.
4. Odpowiedź JSON wraca do frontendu.
5. Frontend renderuje rezultat i aktualizuje stan aplikacji.

Tryby pracy

- Tryb OAuth Spotify: dane pobierane przez access token.
- Tryb demo: dane pobierane z lokalnego zbioru mockData.

Mapa warstw

Warstwa	Technologia	Funkcja
UI	React + Tailwind	Prezentacja danych i obsługa interakcji
API	Express	Logika biznesowa i autoryzacja
ORM	Prisma	Dostęp do bazy PostgreSQL
Integracja	Spotify API	Dane muzyczne i profilowe

[02_backend_rdzen.html](#)

02. Backend rdzen i middleware

Analiza plikow odpowiedzialnych za start serwera i kontrole sesji.

backend/src/index.js

Plik konfiguruje serwer Express i rejestruje route API.

```
app.use('/api/auth', authRoutes)
app.use('/api/user', userRoutes)
app.use('/api/reviews', reviewsRoutes)
app.use('/api/spotify', spotifyRoutes)
app.use('/api/playlists', playlistsRoutes)
app.use('/api/recommendations', recommendationsRoutes)
```

Plik zawiera rowniez endpoint health check oraz middleware globalnej obslugi bledow.

backend/src/middleware/auth.js

Middleware realizuje walidacje tokenu JWT i odswiezanie sesji.

```
const token = req.cookies.token || req.headers.authorization?.split(' ')[1]
const decoded = jwt.verify(token, process.env.JWT_SECRET)
```

```
req.userId = decoded.userId  
req.spotifyAccessToken = decoded.spotifyAccessToken
```

Po wykryciu wygasniecia tokenu wykonywany jest proces refresh token przez Spotify.

backend/src/lib/prisma.js i backend/src/lib/spotify.js

- prisma.js: singleton PrismaClient.
- spotify.js: OAuth, wyszukiwanie, top tracks, audio features, playlist sync.

[03_backend_route_api.html](#)

03. Backend route i API

Opis endpointow oraz mapowania ich do plikow route.

Auth: backend/src/routes/auth.js

- rejestracja i lokalne logowanie,
- logowanie Spotify OAuth,
- demo-login, refresh, logout, endpoint me.

Spotify: backend/src/routes/spotify.js

- search, track details, audio features, top tracks, top artists, genres.
- obsluga trybu demo przez mockData.

Reviews, Playlists, Recommendations

- reviews.js: tworzenie, odczyt i usuwanie recenzji.
- playlists.js: zarzadzanie playlista i synchronizacja do Spotify.
- recommendations.js: podobne utwory i odkrywanie po gatunku.

```
await prisma.review.upsert({  
  where: { userId_trackId: { userId: req.userId, trackId } },  
  update: { rating, content },  
  create: { userId: req.userId, trackId, rating, content }  
})
```

[04_baza_danych_prisma.html](#)

04. Baza danych i Prisma

Opis modeli danych oraz relacji stosowanych w projekcie.

Zrodlo konfiguracji

Plik: backend/prisma/schema.prisma

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

Modele

- User: profil, preferencje, relacje do recenzji i playlist.
- Review: ocena i opis utworu, dane audio, relacja do usera.
- Playlist oraz PlaylistTrack: struktura playlist i kolejnosc utworow.
- LikedTrack: zapis utworow oznaczonych przez uzytkownika.

Ograniczenia i spojnosc

- unikalnosc recenzji user-track.
- unikalnosc tracka w jednej playliscie.
- relacje z kasowaniem kaskadowym.

[05_frontend_rdzen.html](#)

05. Frontend rdzen i komunikacja

Opis plikow odpowiedzialnych za uruchomienie frontendu i komunikacje z API.

frontend/src/main.tsx

Plik inicjuje aplikacje oraz QueryClientProvider.

```
export const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 1000 * 60 * 5, retry: 1 } }
})
```

frontend/src/App.tsx

Plik definiuje routing i warstwe chronionych tras przez ProtectedRoute.

frontend/src/contexts/AuthContext.tsx

Context utrzymuje stan sesji i profil zalogowanego uzytkownika.

frontend/src/lib/api.ts

Wspolny klient HTTP (axios) z interceptor 401 i mechanizmem odswiezenia tokenu.

```
if (status === 401 && !originalRequest._retry) {  
  originalRequest._retry = true  
  await api.post('/auth/refresh')  
  return api(originalRequest)  
}
```

[06_frontend_strony_ui.html](#)

06. Frontend strony i komponenty

Opis logiki ekranow i komponentow interfejsu.

Strony aplikacji

- Home i Login: uruchomienie sesji Spotify lub demo.
- Dashboard: top tracks, top artists, statystyki.
- Search: wyszukiwanie utworow i prezentacja listy.
- Track: szczegoly utworu, recenzje, audio features.
- Reviews: przeglad i usuwanie recenzji usera.
- PlaylistGenerator: generowanie playlisty po gatunku lub tracku.
- Profile: dane profilu i preferencje gatunkowe.
- Community: wyszukiwanie i podglad profili publicznych.

Komponenty wspolne

- Layout: nawigacja i kontener widokow.
- ProtectedRoute: kontrola dostepu do tras.
- StarRating: obsluga oceny co 0.5.
- AudioFeaturesDisplay: wizualizacja cech audio utworu.
- TrackCard: karta prezentacji utworu.

[07_uruchomienie_bezpieczenstwo_testy.html](#)

07. Uruchomienie i analiza techniczna

Sekcja zawiera konfiguracje startowa oraz opis mechanizmow stabilnosci dzialania.

Uruchomienie backend

```
cd backend
npm install
npm run db:generate
npm run db:push
npm run dev
```

Uruchomienie frontend

```
cd frontend
npm install
npm run dev
```

Konfiguracja backend .env

```
PORT=3001
CLIENT_URL=http://localhost:5173
DATABASE_URL=postgresql://USER:PASS@localhost:5432/waveeProjectBW
JWT_SECRET=sekret
SPOTIFY_CLIENT_ID=...
SPOTIFY_CLIENT_SECRET=...
SPOTIFY_REDIRECT_URI=http://localhost:3001/api/auth/callback
```

Mechanizmy dzialania

- Walidacja danych wejsciowych przez Zod.
- Autoryzacja przez JWT i middleware auth.
- Obsluga sesji z odswiezaniem tokenu.
- Tryb demo oparty o mockData przy braku tokenow Spotify.

Podpis autorow dokumentacji: Aleksander Baran, Franciszek Wawrze

Data: 17.03.2026